dSTORMQuant: A Python Package for Post-Processing and Quantitative Analysis of SMLM datasets

Suraj Karki[1], Britta Nemeita[1], Anne Sophie Hammann[1], Sven Thoms[*1]

Affiliations

[1] Biochemistry and Molecular Medicine, Medical School OWL, Bielefeld University

* Corresponding author: sven.thoms@uni-bielefeld.de

Document version: 10th November 2026

Document overview

Program Summary

Program title: dSTORMQuant
Version: 0.0.1
Language: Python, C++
Operating systems: Linux, macOS, and Windows.
License: GNU General Public License v3.0 (GPL-3.0)
Purpose: Algorithms for post-processing and quantitative analysis of SMLM datasets.

Scientific field

Computational analysis of super-resolution microscopy data, focused on Single Molecule Localisation Microscopy, including analysis of spatial molecule patterns, clustering of molecules, colocalisation of different molecules, and nearest-neighbour distance statistics.

Availability and Implementation

dSTORMQuant is implemented primarily in Python with performance-critical clustering components in C++. It is distributed as open source and supports command-line execution with configurable YAML-based workflows. The source code is freely available from the GitHub repository: https://github.com/BCMM-Bielefeld-University/dSTORMQuant

Keywords

dSTORMQuant; SMLM; super-resolution microscopy; clustering; colocalisation; drift correction; spatial analysis

Contents

1. Introduction 4
    1.1 What is dSTORMQuant? 4
    1.2 What dSTORMQuant can do 4
2. Installation of dSTORMQuant 5
    2.1 Requirements 5
    2.2 Installation 5
    2.3 Available Scripts 6
    2.4 The Philosophy 6
    2.5 The Input File Format 7
3. Pipeline Design 8
    3.1 Modularity 8
    3.2 Configuration-driven Behaviour 8
    3.3 Metadata-driven Workflow 8
    3.4 High-level Structure 8
    3.5 Execution Order and Data Dependencies 8
4. Data Flow and Configuration 9
    4.1 Input Data Requirements 9
    4.2 Metadata-driven Workflow 11
    4.3 Configuration Management 13
    4.4 Configuration Description 13
5. Implementation 19
    5.1 Data Loading and Channel Extraction 19
    5.2 Drift Correction: Adaptive Intersection Maximization (AIM) 19
    5.3 Filtering 23
    5.4 Temporal Grouping 24
    5.5 Cell Detection (Optional) 25
    5.6 Nearest Neighbour Analysis (Optional) 26
    5.7 Clustering 27
    5.8 KNN Analysis of Cluster Centroids (Optional) 29
    5.9 Colocalization Analysis 30
    5.10 Relative Enrichment (RE) 30

5.11 Summary Statistics and Reporting 31
5.12 Export of Results 31
5.13 Output Layout (per experiment) 32
6. Post-processing Scripts 33
6.1 merge-csv 33
6.2 save-napari-visualizations 33
6.3 visualization-steps 33
7. References 34

1. Introduction

dSTORMQuant is a modular toolbox of post-processing and analysis algorithms for super-resolution microscopy data, including PALM, STORM, and dSTORM. It is implemented primarily in Python, with performance-critical components in C++ and optional Cython bindings. The project is distributed under the GNU General Public License v3.0 (GPL-3.0) and is intended to make reproducible, configurable dSTORM data analysis accessible to researchers and developers. dSTORMQuant takes localization coordinates and associated metadata as input and focuses on coordinate-based quantitative analysis rather than raw-image fitting.

1.1 What is dSTORMQuant?

Single-molecule localization experiments produce lists of detected emitters (x, y, frame, intensity, precision, and related measurements). These coordinate lists provide the basis for quantitative analyses beyond image rendering. dSTORMQuant provides a pipeline and a collection of modular algorithms that operate on these lists to extract biologically and physically meaningful measures, including localization-precision estimates, spatial point-pattern statistics, cluster identification and characterisation, temporal grouping of repeated detections, colocalization metrics, nearest-neighbour analyses, and cell-level segmentation and phenotyping. The codebase combines high-level Python orchestration with a C++-accelerated clustering implementation for larger datasets.

1.2 What dSTORMQuant can do

- Filter localizations by sigma, intensity, localization uncertainty, and p-value.
- Optionally recalculate localization uncertainty using the formulations proposed by Mortensen et al. [11] at each major pipeline step. Store the results in a separate `LP_calculated` column while preserving the original localization-precision column.
- Correct experimental drift using the Adaptive Intersection Maximization (AIM) algorithm and segment-based validation.
- Merge temporally close localizations using temporal grouping.
- Detect cell boundaries using grid-based or Voronoi-based approaches and extract per-cell metrics.
- Identify and characterise clusters using DBSCAN, HDBSCAN, and FINDER.
- Perform colocalization analysis using coordinate-based colocalization (CBC) and relative-enrichment measures.
- Run nearest-neighbour analyses to characterise spatial organisation, assess inter-channel relationships, and summarise cluster-level relationships.
- Generate standardised quantitative outputs and visualizations using Napari integration and plotting utilities for downstream statistical analysis.
- Pipeline configuration and execution are controlled through a YAML configuration file (`config.yaml`) and command-line entry points.

1.2.1 What dSTORMQuant cannot do

dSTORMQuant is not a localization engine. It is designed to post-process single-molecule localization microscopy (SMLM) data and extract quantitative information. It is not a graphical user interface (GUI) application; it is a Python package that analyses localization coordinates from STORM, dSTORM, and PALM datasets. Its focus is quantitative analysis of super-resolution microscopy data, including spatial point-pattern analysis, clustering, colocalization, and nearest-neighbour statistics.

2. Installation of dSTORMQuant

2.1 Requirements

- Python 3.10 or higher
- Windows, macOS, or Linux

2.2 Installation

2.2.1 Clone git repository

- Install Git (https://git-scm.com/) if necessary. Open a terminal (Linux or macOS) or Command Prompt/PowerShell (Windows), then use `cd` to navigate to the target directory (for example, `cd git`).
- Run: `git clone https://github.com/BCMM-Bielefeld-University/dSTORMQuant.git`

2.2.2 Install the package

- Navigate to the project directory: `cd dSTORMQuant`
- Recommended: Create and activate a virtual environment to isolate the project dependencies.
  - Create the environment: `python -m venv .venv`
  - Activate the environment:
- Windows (PowerShell): .\.venv\Scripts\Activate.ps1
- Linux/macOS: source .venv/bin/activate
- Install the package in editable mode: `pip install -e .`

2.2.3 Build and install FINDER (optional)

- Change directory: cd finder_cpp
- Install build requirements: pip install -r requirements-build.txt
- Build: pip install -e . --no-build-isolation
- Go back: cd ..

2.2.4 Run dSTORMQuant

After a successful build and installation, verify the command by running `dSTORMQuant`.

2.3 Available Scripts

The following CLI commands are supported when executed from the project directory.

Command	Purpose
dSTORMQuant	Main entry command for the SMLM analysis pipeline. Runs the full processing workflow based on config/config.yaml.
merge-csv	Merge CSV outputs from multiple experiments into data/output/merged_channel_data/ grouped by cell line and channel.
save-napari-visualizations	Batch-save Napari views as images into each experiment's test_images/ folder.
visualization-steps	An interactive, terminal-based menu allows users to browse and inspect CSV files from each pipeline stage step by step within an output folder.

2.4 The Philosophy

All data-processing steps in dSTORMQuant follow a staged workflow. First, the input file is validated and the relevant experimental metadata are loaded. The localization coordinates are then partitioned into channel-specific subsets according to the metadata and user-defined configuration. These subsets are processed using the selected analysis modules, including filtering, drift correction, temporal grouping, cell-boundary detection, clustering, colocalization, and nearest-neighbour analysis. This workflow supports quantitative analysis of localizations within detected cellular regions.

Algorithm:

1. Load the configuration file (C).
2. Load the metadata table (M) and select the row corresponding to the current input file.
3. Load the single-molecule localization list (L).
4. Validate the required localization columns and metadata fields.
5. Partition (L) into channel-specific localization streams using the frame and channel ranges defined in the metadata.
6. If lp_calculation.use is enabled, calculate localization precision according to Mortensen et al. [11] for the channel-extracted data. Store the calculated values in LP_calculated and update the per-channel CSV files with the required columns for AIM processing.
7. Apply AIM drift correction to the channel-specific data. If LP calculation is enabled, recalculate LP_calculated on the drift-corrected data.
8. Apply the configured filtering pipeline using the specified threshold parameters. If LP calculation is enabled, recalculate LP_calculated on the filtered data.
9. Apply temporal grouping and duration filtering according to the configuration. If LP calculation is enabled, recalculate LP_calculated on the resulting data.
10. If cell boundary detection is enabled, perform cell boundary detection and recalculate LP_calculated on the resulting data, provided LP calculation is enabled.
11. If enabled, compute nearest-neighbor statistics.

12. If enabled, perform clustering on the processed localization data.
13. If enabled, compute colocalization metrics for the applicable channels.
14. Generate visualizations and summary plots.
15. Save the processed localization data and associated result files.
16. Archive the experiment outputs in a standardised final-results directory structure.

2.5 The Input File Format

For compatibility and consistency across experimental setups, dSTORMQuant uses a structured CSV format for localization data. The format accommodates typical outputs from modern SMLM acquisition and localization software, including CODI, and contains the essential measurements required for spatial, temporal, and photometric analyses.

Each input file is a comma-separated table in which every row represents a single localization event. The format encodes six core observables: the position in the x-dimension, the position in the y-dimension, the acquisition frame index (time of detection), the integrated photon counts of the point spread function (PSF), the channel identifier, and the PSF width estimates in x and y. Additional qualit metrics — localization precision, background density, and statistical p-value — are optional but enable the full filtering pipeline when present. The complete column specification is given in Table 2.

All spatial coordinates are expressed in nanometres. Required column names must match those listed in Table 2 exactly; the pipeline checks them when loading each file and skips any file that lacks a required column. The pipeline supports 2D data only. If a `z (nm)` column contains non-zero numeric values, processing of that file stops; if the column is empty or contains only zeros, it is removed and a warning is logged. Before processing data exported from software with different column conventions, rename the headers to match the names specified here.

3. Pipeline Design

3.1 Modularity

Each stage in the data processing pipeline, such as drift correction, filtering, and clustering, is encapsulated within a module. This ensures that each step can be understood, tested, and modified independently, while the overall workflow remains stable.

3.2 Configuration-driven Behaviour

All parameters that can be adjusted within the data processing pipeline have been centralised within a single YAML configuration file. This ensures that there are no thresholds or constants hard-coded within the data pipeline's logic, thus supporting reproducibility and batch execution with different configurations.

3.3 Metadata-driven Workflow

Instead of deducing the relevant experimental data from file names and the structure of the folders, the information is kept in a separate Excel file, referred to as the metadata file.

3.4 High-level Structure

The pipeline is orchestrated by a central core that coordinates processing modules. Processing modules transform the data at each step; analysis modules compute quantitative metrics; visualization produces plots and figures; config/utils provide configuration, I/O, and cross-cutting concerns.

3.5 Execution Order and Data Dependencies

For each input file, the pipeline executes the following steps in strict sequence. Later steps depend on the outputs of earlier ones:

Table 1: Execution Order of the Modules

Step	Module	Description	Data Dependency
1	main.py	Load configuration and metadata; discover and iterate over input files	None
2	pipeline.py	Load raw CSV, validate columns, extract channels, create per-channel files	Raw CSV, metadata
3	visualization	Initial visualizations and summary metrics	Loaded / channel-extracted data
4	drift_correction	AIM drift correction per channel; align Ch2 to Ch1 drift for dual-channel	Per-channel CSVs
5	filtering	Sigma, photon count, localization precision, p-value filters	Drift-corrected data
6	temporal_grouping	Merge repeated localizations into tracks	Filtered data

7	cell_detection	Optional cell boundary detection	Temporal-grouped data
8	nearest_neighbor_ analysis	KNN distance analysis (optional)	Cell-detected (or grouped) data
9	clustering	DBSCAN, HDBSCAN, or FINDER clustering per channel	Cell-detected (or grouped) data
10	clustering	Optional cluster-centre KNN	Cluster stats
11	colocalization	CBC and RE colocalization (dual-channel only)	Cell-detected (or grouped) dual-channel data
12	pipeline.py	Export CSVs and images; create stacked histograms; zip outputs	All intermediates

4. Data Flow and Configuration

This section defines the input data format, the metadata-driven workflow for file and channel detection, and the configuration management system. Correct setup of input data and metadata is a prerequisite for the successful execution of the pipeline.

4.1 Input Data Requirements

4.1.1 Format and Column Semantics

The pipeline expects the localization points in the form of CSV files with the following columns:

Table 2: Description of the Columns in Input CSV

Column	Semantics	Unit	Mandatory
x (nm)	X-coordinate of a localised molecule	nanometres	Yes
y (nm)	Y-coordinate of a localised molecule	nanometres	Yes
z (nm)	Z-coordinate of localized molecule	nanometres	No
sigmaX (nm)	Estimated PSF width in x	nanometres	No
sigmaY (nm)	Estimated PSF width in y	nanometres	No
intensity (photons)	Total photon counts for the localization	photons	No
background (photons/nm^2)	Background photon density	photons/nm²	No
p-value	Statistical significance of the localization fit	0–1	No
localization precision (nm)	Estimated positional uncertainty	nanometres	No
channelIndex	Channel identifier (integer)	0-3	Yes
frameIndex	Frame number during acquisition	integer	Yes
sigma [nm]	Single PSF width (alternative to sigmaX/sigmaY); used by LP recalculation when X/Y are absent	nanometres	No

These columns match the standard output format of dSTORM localization software. The input data must be a 2D localization table with the columns listed in Table 2. The pipeline only works with 2D data and does not support 3D (z) coordinates. The column names must match exactly as specified. Optional quality columns, such as sigmaX, sigmaY, intensity, background, localization precision (nm), and p-value, are specified in data.input.optional_quality_columns. If an optional column is missing, the corresponding filter is skipped, a warning is logged, and the action is recorded in filtering_report.json. If a required column is missing, processing of the affected file is stopped, while the remaining files in the batch continue to be processed. The current release supports 2D data only. If the z (nm) column contains non-zero numeric values, processing of the affected file is stopped. If the column is empty or contains only zeros, it is removed, and a warning is logged. If a channelName column is present, it is removed during data loading. Before placing files in data/input/, rename the export column headers to match the documented NimOS-style names.

4.1.2 Co-ordinate and Quality Metric Conventions

- Coordinates: All spatial coordinates are in nanometres. The pipeline assumes a flat 2D field of view. The pixel size in the config is used only during drift correction and localization precision calculation for the transformation between pixel and nm units.
- Localization precision is commonly estimated using $\sigma_{loc} = \sqrt{((\sigma^2 + (a^2/12))/N + (8\pi\sigma^4 b^2)/(a^2 N^2))}$, where σ is the PSF width, a is the pixel size, N is the photon count, and b is the background [2]. In this guide, the reported value is treated as positional uncertainty: a larger value indicates lower localization precision and reliability. When `lp_calculation.use` is enabled, dSTORMQuant recalculates localization precision using the formulations proposed by Mortensen et al. [11]. The user can select either the least-squares (LS) formulation or the maximum-likelihood estimation (MLE) formulation.

Least-squares formulation
The localization variance (Varx) is calculated as

$$F\frac{s_a^2}{N}\left(\frac{16}{9}+\frac{8\pi s_a^2 b^2}{Na^2}\right) \quad (1)$$

Maximum-likelihood formulation
The localization variance (Varx) is calculated as

$$F\frac{s_a^2}{N}\frac{1}{-\int_0^1 \frac{tln(t)}{t+\rho}\,dt} \quad (2)$$

The integration factor (ρ) and the effective Gaussian standard deviation (s_a) are defined as

$$\rho = \frac{2\pi s_a^2 b^2}{N a^2}, s_a = \sqrt{s^2 + \frac{a^2}{12}} \quad (3)$$

The localization precision is obtained as the square root of the estimated localization variance:

$$\sigma_{\text{loc}} = \sqrt{\text{Var}(x)} \quad (4)$$

Here, (N) denotes the photon count for the localization, (s) is the Gaussian point-spread function (PSF) standard deviation, (a) is the pixel size in nanometres, and (b^2) represents the expected background photon contribution per pixel. The factor (F) accounts for camera excess noise: (F=2) for an EMCCD camera when the option is enabled and (F=1) otherwise.

For each localization, dSTORMQuant uses its own intensity, background, and Gaussian-width information. The pixel size is obtained from drift_correction.pixel_size. If both sigma_x and sigma_y are available in the input CSV, they are combined according to the configured sigma_mode (mean, geo, x, or y). Otherwise, the available single sigma column is used.

The recalculated precision values are stored in LP_calculated, leaving the original localization-precision column (lp) unchanged. The existing localization-precision filtering step continues to use the original lp values, including after drift correction.

- P-value: Lower values indicate stronger evidence that the localization is a real emitter rather than noise. Typical thresholds range from 0.01 to 0.1.

4.2 Metadata-driven Workflow

4.2.1 Purpose of the Metadata File

An Excel metadata file located at `data/metadata/<xlsx_filename>` controls input-file discovery and channel configuration. This design supports:

- Flexible file naming: The pipeline matches input CSVs to metadata rows by base name (without extension). File names need not encode channel information.
- Per-experiment channel mapping: Different experiments may use different channel indices or frame ranges; the metadata specifies these per file.
- Scalability: Adding new experiments or conditions only requires updating the metadata and placing CSVs in the input directory.

4.2.2 Required and Optional Metadata Columns

The Excel metadata file must use normalised column names in lowercase, without whitespace.

- file_name: Base name of the input CSV (e.g., 'AHA_24014_U2OS-WT_AST0126_AST0063_posXY0_channels_t0_posZ0')
- first_channel_index: Integer channel index for the first acquired channel
- first_ch_frame_last: Last frame number for channel 1
- second_channel_index: Channel index for the second channel
- second_ch_frame_last: Last frame for channel 2

Optional columns:

Optional metadata columns are configured through data.input.optional_metadata_columns in config.yaml. The default column names are listed below.

- experiment_number: Experiment identifier
- initials: Initials associated with the experiment
- tag: Additional experiment identifier or label
- first_channel_label: Optional display label for the first channel
- second_channel_label: Optional display label for the second channel

The names of optional columns can be customized through the configuration file.

Channel labels, such as AST0063 and PST0126, can be extracted from the input filename using extract_channels_from_filename(). These labels are used for filtering and output labelling. If first_channel_label or second_channel_label is provided in the metadata, the metadata label is preferred; otherwise, the label may be resolved from the filename.

4.2.3 Frame Ranges and Channel Configurations

The pipeline does not require a fixed acquisition length, such as 10,000 or 20,000 frames. Instead, the frame ranges for each input file are determined by the last-frame values specified in the metadata Excel file.

Initial frame exclusion: To account for activation or burn-in at the beginning of an acquisition, the first skip_frames frames of each channel are excluded. This parameter is configured through data.input.skip_frames in config.yaml and defaults to 200.

Dual-channel frame handling: In dual-channel mode, channel 2 is treated as starting immediately after channel 1 ends. Its frame range is therefore offset by the last-frame value of channel 1. The pipeline validates that the channel frame ranges are positive and correctly ordered.

Single-channel and dual-channel detection: The processing mode is determined from the metadata row:

- Dual-channel mode: Both second_channel_index and second_ch_frame_last are provided. The pipeline processes both channels and applies the corresponding channel-specific operations.
- Single-channel mode: The second-channel index and last-frame fields are blank. The pipeline processes only the first channel, writes only the *_ch1.csv file, and skips channel 2 drift transfer.

4.3 Configuration Management

All tunable parameters are available in **config/config.yaml.** The configuration is:

- Loaded once at pipeline startup via **load_config()** in **core/config/loader.py.**
- Validated using Pydantic models in **core/config/models.py** — invalid types or out-of-range values trigger clear errors before any processing begins.
- Passed through to **process_single_file()** and from there to each step function.

4.4 Configuration Description

4.4.1 Data Input (data)

Key	Type	Description	Example
data.input.file_format	string	Format of raw localization files	csv (only supported format)
data.input.required_columns	list[string]	Column names required in input CSV	"x (nm)", "y (nm)", "channelIndex", "frameIndex"
data.input.xlsx_filename	string	Metadata Excel filename (placed in data/metadata/)	"SMLM Data_Input.xlsx"
data.input.skip_frames	number	Number of frames to skip at the start of each channel's frame range	e.g. 200
data.input.required_metadata_columns	list[string]	Column names required in metadata xlsx	file_name, first_channel_index, first_ch_frame_last, second_channel_index, second_ch_frame_last
data.input.optional_quality_columns	list[string]	Documented optional quality CSV headers (filters skip if absent)	sigmaX (nm), intensity (photons), localization precision (nm), …

data.input.optional_metadata_columns	dict[string,string]	Optional metadata Excel header map	experiment_number, initials, tag, first_channel_label, second_channel_label

4.4.2 Channel Display (channels)

Key	Type	Description	Example
channels.Ch1.color	string	Hex colour for Channel 1	"#FF00FF"
channels.Ch2.color	string	Hex colour for Channel 2	"#00FFFF"

4.4.3 Visualization

Key	Type	Description	Example
visualization.use_napari	boolean	If true, save Napari point/cluster preview PNGs; if false, skip those (headless/cloud). Matplotlib histograms from plot_metrics still run.	e.g. true or false

4.4.4 Filtering

Key	Type	Description	Example
filtering.intensity.min_value	number	Intensity threshold value	e.g. 300
filtering.localization_precision.threshold_value	float	Maximum allowed localization-uncertainty value (nm)	e.g. 20.0
filtering.p_value.threshold_value	float	Max allowed p-value	0–1
filtering.sigma.min_value	float	Min allowed σ (nm)	e.g. 50.0
filtering.sigma.max_value	float	Max allowed σ (nm)	>min_value
filtering.sigma.use	boolean	Enable/disable sigma filter	true/false
filtering.intensity.use	boolean	Enable/disable intensity filter	true/false
filtering.localization_precision.use	boolean	Enable/disable localization-precision filter	true/false
filtering.p_value.use	boolean	Enable/disable p-value filter	true/false

4.4.5 Drift Correction (drift_correction)

Key	Type	Description	Example
drift_correction.pixel_size	number	Pixel size (nm)	e.g. 117

drift_correction.segmentation	integer	Temporal segments	e.g. 100
drift_correction.intersect_d_nm	float	Intersection distance (nm)	e.g. 20
drift_correction.roi_r_nm	float	ROI radius (nm)	e.g. 60
drift_correction.drift_validation.use	boolean	Enable drift validation	true or false
drift_correction.drift_validation.max_segment_drift_nm	float	Max allowed segment drift (nm)	e.g. 35.0
drift_correction.drift_validation.n_previous_segments	integer	Previous segments used for replacement	e.g. 10
drift_correction.sanity_checks.filter_boundary	boolean	Remove x ≤ 0 or y ≤ 0	true or false
drift_correction.sanity_checks.filter_parameters	boolean	Remove sx ≤ 0, sy ≤ 0, lp ≤ 0	true or false
drift_correction.sanity_checks.remove_invalid_values	boolean	Remove localizations with inf/nan in core fields (defaults to true if omitted)	true/false
drift_correction.aim_options.fft_peak_scale	string	Chooses how the AIM algorithm converts the FFT correlation peak into a drift distance in camera pixels.	search_diameter or intersect_d
drift_correction.aim_options.exclude_self_round2	boolean	Exclude the active segment from the round-2 reference during alignment	true/false
drift_correction.aim_options.drift_reference	string	frame0 sets drift to zero at the first frame, while mean centres the drift curve on its mean.	frame0 or mean

4.4.6 Localization Precision Recalculation (lp_calculation)

Key	Type	Description	Example
lp_calculation.use	boolean	Enable Mortensen LP recalculation	true/false
lp_calculation.method	string	Mortensen formula	"mle" (Eq. 1) or "lsq" (Eq. 2)

lp_calculation.emccd	boolean	Apply EMCCD excess-noise factor 2 on variance	true/false
lp_calculation.bg_unit	string	Units of the background column	"per_nm2" or "per_pixel"
lp_calculation.sigma_mode	string	How to form “s” from sigma_x/sigma_y when both exist: mean=(σx+σy)/2; geo=√(σx·σy); x=use σx only; y=use σy only (ignored for a single sigma column)	"mean", "geo", "x", or "y"
lp_calculation.output_column	string	Name of recomputed precision column	"LP_calculated"
lp_calculation.columns.intensity	string	Exact intensity/photons CSV header	"intensity (photons)"
lp_calculation.columns.background	string	Exact background CSV header	"background (photons/nm^2)"
lp_calculation.columns.sigma	string	Single-sigma header (fallback if X/Y absent)	"sigma [nm]"
lp_calculation.columns.sigma_x	string	Sigma X header (preferred with sigma_y when both present)	"sigmaX (nm)"
lp_calculation.columns.sigma_y	string	Sigma Y header (preferred with sigma_x when both present)	"sigmaY (nm)"

4.4.7 Temporal Grouping (temporal_grouping)

Key	Type	Description	Options
temporal_grouping.use	boolean	Enable temporal grouping	true/false
temporal_grouping.max_frame_gap	integer	Max frame gap	e.g. 2
temporal_grouping.max_distance_nm	float	Max linking distance (nm)	e.g. 50
temporal_grouping.min_duration	integer	Min track duration	e.g. 1
temporal_grouping.max_duration	integer	Max track duration	≥ min_duration

4.4.8 Cell Detection (cell_detection)

Key	Type	Description	Options
cell_detection.use	boolean	Enable cell detection	true/false
cell_detection.approach	string	Algorithm	"grid" or "vornoi"
cell_detection.grid.grid_size	integer	Grid bins per axis	e.g. 100
cell_detection.grid.high_density_threshold	integer	Minimum localizations per bin	e.g. 3

cell_detection.vornoi.percentile	float	Percentile before Otsu threshold	1–100

4.4.9 Nearest Neighbour Analysis (nearest_neighbor_analysis)

Key	Type	Description	Options
nearest_neighbor_analysis.use	boolean	Enable NN analysis	true/false
nearest_neighbor_analysis.radius	float	Radius (nm)	e.g. 100
nearest_neighbor_analysis.k	integer	Number of neighbours	e.g. 1
nearest_neighbor_analysis.algorithm	string	sklearn algorithm	"auto", "ball_tree", "kd_tree", "brute"
nearest_neighbor_analysis.metric	string	Distance metric	"euclidean"

4.4.10 Clustering

Key	Type	Description	Options
clustering.use	boolean	Enable clustering	true/false
clustering.use_cluster_knn	boolean	Enable cluster centroid KNN	true/false
clustering.ch1.use	boolean	Whether to run clustering on ch1 or not	true/false
clustering.ch1.method	string	Clustering method (Ch1)	"dbscan", "hdbscan", "finder"
clustering.ch1.dbscan.eps	float	DBSCAN radius (Ch1)	e.g. 100
clustering.ch1.dbscan.min_samples	integer	DBSCAN min samples (Ch1)	e.g. 10
clustering.ch1.hdbscan.min_cluster_size	float	Minimum size of cluster	e.g. 10
clustering.ch1.hdbscan.min_samples	integer	HDBSCAN minimum samples (Ch1)	e.g. 10
clustering.ch1.finder.threshold	integer	Minimum points for cluster	e.g. 10
clustering.ch1.finder.points_per_dimension	integer	Grid resolution for parameter search	e.g. 15
clustering.ch1.finder.algorithm	list[string]	Inner algorithm	e.g. "dbscan", "DbscanLoop"
clustering.ch1.finder.min_threshold	integer	Minimum threshold to search	e.g. 5
clustering.ch1.finder.max_threshold	integer	Maximum threshold to search	e.g. 21
clustering.ch1.finder.decay	float	Decay for parameter search	e.g 0.5
clustering.ch2.use	boolean	Whether to run clustering on ch2 or not	true/false
clustering.ch2.method	string	Clustering method (Ch2)	Same as ch1

clustering.ch2.dbscan.eps	float	DBSCAN radius (Ch2)	e.g. 100
clustering.ch2.dbscan.min_samples	integer	DBSCAN min samples (Ch2)	e.g. 10
clustering.ch2.hdbscan.min_cluster_size	float	Minimum size of cluster	e.g. 10
clustering.ch2.hdbscan.min_samples	integer	HDBSCAN minimum samples (Ch2)	e.g. 10
clustering.ch2.finder.threshold	integer	Minimum points for cluster	e.g. 10
clustering.ch2.finder.points_per_dimension	integer	Grid resolution for parameter search	e.g. 15
clustering.ch2.finder.algorithm	list[string]	Inner algorithm	e.g. "dbscan", "DbscanLoop"
clustering.ch2.finder.min_threshold	integer	Minimum threshold to search	e.g. 5
clustering.ch2.finder.max_threshold	integer	Maximum threshold to search	e.g. 21
clustering.ch2.finder.decay	float	Decay for parameter search	e.g 0.5

4.4.11 Colocalization

Key	Type	Description	Options
colocalization.cbc.radius	float	CBC radius (nm)	e.g. 100
colocalization.cbc.n_steps	integer	Radius steps	e.g. 10
colocalization.use	boolean	Enable/disable all colocalization analyses (CBC and RE)	true/false

5. Implementation

This section provides a detailed description of each processing step: the algorithms used, the reason for design choices, key parameters, and expected outputs.

5.1 Data Loading and Channel Extraction

5.1.1 What happens in this step?

- Load the complete CSV using `load_data()` from `utils/data_handling.py`. The raw CSV contains localization coordinates from all channels and frames.
- Validate required columns: If any column from config.data.input.required_columns is missing, the pipeline logs an error and skips the file.
- Look up metadata: The metadata row for this file is retrieved by matching the base name to the file name column.
- The code filters localizations for each channel by selecting rows where channelIndex matches and frameIndex is within a specific range, from the channel's start frame plus skip_frames (default 200, set in config.yaml) up to the last frame for that channel. This produces separate DataFrames for each channel.
- Save per-channel CSVs: Each channel subset is written to *_ch1.csv and *_ch2.csv in the input directory. These temporary files are used by the AIM drift correction module, which expects a single-channel CSV per run. For single-channel experiments only *_ch1.csv is written. Temporary channel CSVs under data/input/ are removed after drift correction.
- Combine channels: For downstream steps, channel-specific data are concatenated into a single DataFrame. The `channelIndex` column is preserved so that each step can operate on individual channels when needed.

5.1.2 Why is it designed this way?

Drift correction is applied independently to each channel because the channels are acquired sequentially and may experience different drift during their respective acquisition windows. For dual-channel data, the pipeline then aligns channel 2 with channel 1 by applying the documented channel-1 drift offset to the channel-2 coordinates. This places both channels in a common coordinate system for colocalization and other inter-channel analyses.

5.2 Drift Correction: Adaptive Intersection Maximization (AIM)

Due to mechanical drift of the stage or sample during data acquisition, localizations shift over time. If uncorrected, this drift can blur the reconstructed image and negatively affect spatial analysis. AIM corrects this drift by identifying stable reference points—localizations that appear in multiple temporal segments—and estimating the displacement between these segments. The method does not require fiducial markers or separate reference channels. The implementation used here is adapted from the AIM method implemented in Picasso [3] and modified for pipeline integration, including CSV input/output handling, sanity checks, drift validation, and HDF5 conversion for internal processing.

5.2.1 Algorithm

1. Temporal Segmentation: The acquisition is divided into N equal segments (default: N = 100). Each segment contains a subset of frames.
2. Reference Selection: In the first round, the first segment is used as the reference. In the second round, the entire (partially corrected) dataset is used as a reference to refine the first segment as well.
3. Per-segment Drift Estimation:
   - Convert reference and target localizations to a discretised grid in units of "intersect_d" (intersection distance).
   - For each shift within a local search region (ROI) of size "roi_r," calculate the number of overlapping reference and target localizations (occupying the same grid cell).
   - The shift that maximises this overlap is defined as the coarse drift estimate.
   - Apply FFT-based sub-pixel refinement to obtain a precise drift estimate.
4. Interpolation: Cubic spline interpolation (with boundary extension) is used to obtain per-frame drift from the per-segment drift estimates.
5. Coordinate correction: For each localization, subtract the drift corresponding to its frame index.
6. Two-round refinement: The first round corrects drift relative to the initial segment; the second round uses the full dataset as a reference, improving accuracy for the early frames and reducing bias.

5.2.2 AIM Implementation Options

- The AIM module in dSTORMQuant is adapted from the Picasso AIM module [3]. In addition to the shared AIM parameters (pixel size, segmentation, intersection distance, and search radius), three numerical implementation choices are configurable under drift_correction.aim_options. The defaults preserve the existing dSTORMQuant behaviour, while alternative settings allow these choices to follow the conventions of original Picasso AIM. The shared parameters and the overall processing pipeline remain unchanged.
  - **FFT peak scaling (aim_options.fft_peak_scale):** After the coarse overlap search, AIM refines the drift estimate using the FFT peak of a local correlation map. This option determines how the peak displacement is converted into camera-pixel units. The dSTORMQuant default, search_diameter, scales the peak using the search-window diameter (2 * roi_r) divided by the number of correlation bins. The Picasso AIM, instead uses the intersection bin spacing. The former relates the estimated displacement to the physical extent of the search region, whereas the latter follows the scaling convention used in Picasso AIM.
  - **Round-2 self-exclusion (aim_options.exclude_self_round2):** AIM estimates drift in two rounds. During the second round, this option controls whether the temporal segment being aligned is excluded from its reference. The dSTORMQuant default, false, retains the full reference, allowing both rounds to use the same full-reference intersection procedure. Setting it to true adopts the Picasso AIM leave-one-segment-out behavior, in which the segment being aligned is excluded from the reference.

- **Drift reference (aim_options.drift_reference):** This option determines how the final drift curve is anchored. With frame0 (the dSTORMQuant default), the drift of the first frame is subtracted, setting the initial drift to zero and expressing subsequent drift relative to the start of the acquisition. With mean (the Picasso AIM), the mean drift is subtracted, centring the curve around zero. First-frame anchoring provides a consistent reference for dual-channel registration, where the estimated drift from channel 1 is subtracted from channel 2 coordinates, while mean-centering follows the Picasso AIM convention.

- The default and Picasso-compatible AIM profiles are summarised below.

Option	dSTORMQuant AIM	Picasso-compatible AIM
fft_peak_scale	search_diameter	intersect_d
exclude_self_round2	false	true
drift_reference	frame0	mean

5.2.3 Key Parameters

- **pixel_size:** Camera pixel size in nm. Used to convert between pixel and nm units internally. Must match the acquisition setup.
- **segmentation**: Number of temporal segments. More segments yield finer temporal resolution but increase noise; fewer segments are smoother but may miss rapid drift.
- **intersect_d_nm**: Distance (in nm) used for discretising coordinates when counting overlaps. Should be on the order of localization precision (e.g., 20 nm).
- **roi_r_nm**: Radius of the local search region in nm. Must be larger than the expected drift per segment. Too small underestimates drift; too large increases computation and can introduce noise.
- **fft_peak_scale**: Chooses how the AIM algorithm converts the FFT correlation peak into a drift distance in camera pixels.
- **exclude_self_round2**: In the second AIM pass, controls whether the time segment being corrected is left out of the reference image.
- **drift_reference**: Determines how the drift curve is zeroed: frame0 sets the first frame to, while mean centres the curve around its average value.

5.2.4 Drift Validation

Drift validation is an optional quality-control step performed after AIM estimates drift for each temporal segment. Although AIM usually works well, some segments may produce unrealistic jumps due to sparse data, noise, or acquisition glitches. These outliers can affect the interpolated per-frame drift curve and distort the corrected localization coordinates. When drift_correction.drift_validation.use is enabled, the relative drift increments of each segment are checked. Their magnitude in nanometres is calculated as

$$d = \sqrt{(\Delta x)^2 + (\Delta y)^2} \times \text{pixel_size}$$

If this value exceeds max_segment_drift_nm, the segment is treated as an outlier. The check uses relative drift rather than the absolute sample position. Cumulative drift is reconstructed from the corrected relative increments.

Correction is performed sequentially in time. For a flagged segment with preceding segments, the relative (Δx) and (Δy) values are replaced by the mean of up to n_previous_segments preceding segments, including segments that have already been corrected. If the first segment is flagged, its relative drift is set to zero because no preceding history is available.

After all segments have been processed, cumulative drift is recalculated. If any segments were replaced, the cubic-spline interpolation to the per-frame drift curve is also recalculated so that localizations use the corrected trajectory.

The configuration parameters are:

- **use**: Enables or disables drift validation; default is false.
- **max_segment_drift_nm**: Drift-magnitude threshold in nanometres (e.g., 35 nm).
- **n_previous_segments**: Number of preceding segments used to calculate the replacement mean (e.g., 10).

The outputs include *_drift_segments.csv, *_aimdrift.txt, and the drift plot. The log records each replaced segment and the total number of replacements.

Drift validation is useful when the AIM trace contains isolated, implausible spikes. It can remain disabled for smooth traces. A threshold that is too low may remove genuine motion, while a threshold that is too high may leave artefacts uncorrected. Drift validation modifies the drift curve.

5.2.5 Sanity Checks

Sanity checks validate and clean the localization data after drift correction, when AIM saves the drift-corrected table. They do not modify the drift curve; instead, they remove numerically invalid or physically implausible localizations before filtering and subsequent analysis.

Sanity checks are configured under drift_correction.sanity_checks and applied through ensure_sanity when localizations are written during CSV-to-HDF conversion and after AIM processing.

They include three options:

- **remove_invalid_values**: Removes rows containing inf or NaN values in core fields: frame, x, y, sx, sy, lp, photons, bg, pvalue, and channelIndex. Additional columns, such as LP_calculated, are excluded from this check. Therefore, NaN values in these columns do not cause a row to be removed. This check is forced on when saving drift-corrected data, even if it is not specified in the YAML configuration.
- **filter_boundary**: Retains only localizations with (x > 0) and (y > 0). Localizations at or beyond the origin after drift correction are removed as outside the valid image domain.
- **filter_parameters**: Requires sx, sy, and/or lp to be greater than zero when these fields are present. Non-positive values may indicate failed fits or incorrect mappings. Missing fields are skipped for this check.

Removed localizations can be saved to a *_removed_locs.csv file alongside the HDF5 file for debugging. The log records the number of removed localizations and the reasons for their removal. The remaining localizations proceed to filtering, temporal grouping, and subsequent pipeline stages.

5.3 Filtering

Filters are applied step-by-step in a chosen order to remove the most problematic localizations first. Each filter step reduces the dataset; downstream filters operate on the already-filtered result. The applied filters are described below. Each filter also has a use flag (filtering.<step>.use). If use is false, or if the required quality column is absent after drift, that step is skipped. Per-step status is written to filtering_report.json under the filtered output folder.

5.3.1 Sigma Filter

PSF width (σ) represents the quality of the Gaussian fit. Values far from the expected PSF size indicate misfits, de-focus, or non-Gaussian artefacts. The Sigma filter removes localizations with poor PSF fits.

Criterion: sigma_min ≤ σx, σy ≤ sigma_max

5.3.2 Photon Count Filter

Removes noisy localizations by keeping only those with photon counts above a fixed minimum value, regardless of channel type.

5.3.3 Localization Precision Filter

The localization precision refers to the certainty level that we have about the location of a single emitter. This filter will eliminate localizations whose precision is greater than the defined threshold.

5.3.4 P-value Filter

A low p-value indicates stronger evidence that a detection corresponds to a real emitter rather than noise. This filter retains localizations with p-values below the configured threshold.

5.4 Temporal Grouping

In direct stochastic optical reconstruction microscopy (dSTORM), temporal grouping is the post-processing step in which multiple sequential detections (blinks) from the same fluorophore are merged into a single representative coordinate [4]. Without temporal grouping, one molecule may contribute multiple localizations, artificially increasing the estimated density and cluster sizes. The method performs spatiotemporal clustering using fixed thresholds for spatial distance and frame separation [5]. It does not adapt to localization uncertainty; instead, it applies globally defined constraints.

5.4.1 Methodology

1. Spatial Indexing: A cKDTree is constructed over all localization coordinates to enable efficient neighbour queries.
2. Spatiotemporal Clustering: For each unvisited localization, a breadth-first search is performed to identify neighbouring detections that satisfy both:
    - Spatial proximity: within a fixed distance threshold
    - Temporal proximity: within a predefined maximum frame gap

    All detections connected under these constraints form a single group.

3. Group Merging: Each group is merged into a single representative localization using the following pooling strategy:
    - Mean spatial co-ordinates
    - Summed photon counts
    - Aggregated precision and auxiliary parameters
4. Duration-based filtering: Groups can be filtered based on their temporal duration:
    - Exclude short-lived tracks (e.g., noise or transient artefacts)
    - Remove anomalously long tracks

5.4.2 Parameters

- **max_frame_gap**: This is the largest allowed frame difference between two candidate localizations to still consider them part of the same track.
- **max_distance_nm**: This is the fixed spatial radius used to decide whether two localizations are close enough to be linked.
- **max_duration**: Tracks longer than this are removed. This helps reject suspiciously long tracks that might result from over-linking or artefacts.
- **min_duration**: After linking, each track gets a duration measured in frames. Tracks shorter than this are removed.

5.5 Cell Detection (Optional)

Cell detection limits analysis to localizations that lie within cellular regions, reducing the influence of extracellular or non-cellular background. This step is optional and can be skipped so that the entire field of view is used for clustering and colocalization.

5.5.1 Grid-based Cell Detection

Methodology

1. Binning: The field of view is divided into a 2D grid. The number of bins along each axis is set by grid_size, so the grid has grid_size × grid_size cells. Bin edges are defined by linearly spacing between the minimum and maximum x and y in the dataset.
2. Assignment: Each localization is assigned to a bin via digitisation; columns x_bin and y_bin are added. Only rows with valid bin indices (0 to grid_size - 1) are kept.
3. Density: For each bin, the number of localizations is counted. Bins with count ≥ high_density_threshold are classified as high-density (cell region).
4. Filtering: A Boolean column, `is_cell`, is set to `True` for localizations in high-density bins. The output DataFrame contains only rows for which `is_cell == True`.

Parameters

- **grid_size**: Number of bins per axis. Larger values give finer spatial resolution but smaller counts per bin; smaller values give coarser regions.
- **high_density_threshold**: Minimum localizations per bin to consider the bin as part of the cell. Bins below this are treated as background.

5.5.2 Voronoi-based Cell Detection

Methodology

1. Voronoi tessellation: A Voronoi diagram is computed from all localization coordinates (x, y). Each point has an associated Voronoi cell (region of space closer to that point than to any other).
2. Cell area**:** For each finite Voronoi region, the polygon area is computed using the shoelace formula on the region vertices. Infinite or empty regions are excluded. The area is stored in a new column voronoi_area.
3. Clipping**:** To reduce the influence of extreme, often boundary, regions, Voronoi areas are clipped at the percentile (e.g. 99th percentile). Only finite areas ≤ this clip value are used for thresholding.
4. Thresholding**:** Otsu's method is applied to the clipped Voronoi areas to automatically choose a threshold that separates two classes. Points whose Voronoi area is below (or equal to) this threshold are classified as "cell region"; the rest as background.
5. Filtering: A column, `is_cell`, is set accordingly. The returned DataFrame contains only localizations for which `is_cell == True`. The Voronoi object is also returned for optional visualization.

Parameter

- **percentile**: Percentile used to clip the largest Voronoi areas before Otsu thresholding. Reduces the impact of edge and outlier regions.

5.6 Nearest Neighbour Analysis (Optional)

Nearest neighbour analysis quantifies spatial relationships between localizations, either within a single channel (intra-channel) or between two channels (inter-channel). It supports localization-level characterisation (e.g. distance distributions) and is used again at the cluster level when cluster-centre KNN is enabled.

5.6.1 Methodology

- Configuration: The approach is read from config. Parameters k, algorithm, metric, and radius are used for nearest neighbour and mean-distance-within-radius computations.
- Intra-channel Analysis: For each channel (Channel1, Channel2), localizations are subset by channelIndex. For each point, the k nearest neighbours within the same channel are found. Distances are stored; the first nearest neighbour distance is used for histograms and a new column.
- Inter-channel Analysis: For each ordered pair of distinct channels (Ch1→Ch2, Ch2→Ch1), distances from each localization in the "primary" channel to the nearest localization in the "reference" channel are computed. New columns are added; histograms and CSVs are saved.

- Mean distance within Radius: For a fixed radius r, the mean distance from each point to all neighbours within radius r is computed (intra and, for two channels, inter). Results are stored in columns, with corresponding histograms and CSVs.

5.6.2 Parameters

- **use**: whether to run this step or not in the pipeline. e.g. true/false
- **k**: Number of nearest neighbours to compute (e.g., k = 1 for the first nearest neighbour).
- **algorithm**: Search algorithm used by NearestNeighbors. Possible options: auto, ball_tree, kd_tree, or brute.
- **metric**: Distance metric used for neighbour computation (e.g., Euclidean).
- **radius**: Radius (in nanometres) used for computing the mean distance of neighbours within a specified spatial radius.

5.7 Clustering

Spatial clustering groups localizations into molecular groups (e.g. protein clusters). The pipeline supports three methods per channel: DBSCAN, HDBSCAN, and FINDER. Each channel can use a different method. Clustering is run on the same data used for optional cell detection.

5.7.1 Methodology

- Per-channel Processing: For each channel (Ch1, Ch2), localizations are filtered by channelIndex. Coordinates are taken from the x and y columns.
- Method Call:
  - DBSCAN: sklearn.cluster.DBSCAN is fitted with eps (nm) and min_samples. Points not in any cluster receive label -1.
  - HDBSCAN: sklearn.cluster.HDBSCAN is fitted with min_cluster_size, min_samples, and cluster_selection_epsilon. Again, noise is labelled -1.
  - FINDER: The C++ extension finder_cpp.run_finder_2d is called with the channel's finder parameters. FINDER explores a range of (ε, minPts)-like parameters and returns optimal labels plus chosen parameters (e.g. sigma, threshold). These are saved to a JSON file per channel.
- Label Offset: For dual-channel data, cluster IDs of Ch2 are offset so they do not collide with Ch1. Noise remains -1.
- Assignment: The combined DataFrame receives a cluster column. Each row is assigned the cluster label of its channel.
- Cluster Statistics: For each channel, save_cluster_stats() builds a per-cluster table: cluster ID, channel index, channel label, n_points, centroid (x_center, y_center), standard deviations, convex-hull area, and density. Clusters with fewer than three points get an area of 0. A row for "noise" (cluster -1) is appended. The table is saved.
- Outputs: The clustered DataFrame is saved. Cluster stat CSVs and JSON (for FINDER) are written to the clustering output directory. Downstream, histograms of cluster size and area are generated, and summary statistics are appended for reporting.

5.7.2 DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm designed to discover clusters of arbitrary shape in spatial databases [6].

Parameters

- **eps**: Maximum distance (nm) between two points for them to be in the same neighbourhood. Critical for cluster extent.
- **min_samples**: Minimum points in a neighbourhood for a core point. Smaller values yield more, smaller clusters; larger values favor fewer, denser clusters.

5.7.3 HDBSCAN

HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm that builds a hierarchy of clusters based on varying data density and extracts the most stable clusters while identifying outliers [7].

Parameters

- **min_cluster_size**: Minimum number of points in a cluster.
- **min_samples**: Used for core distance / stability.

5.7.4 FINDER

FINDER is an unbiased clustering algorithm for single-molecule localization microscopy that automatically selects global parameters to robustly identify molecular clusters while minimizing false positives and false negatives in noisy and densely populated data [8].

Algorithm

1. Parameter Search**:** A grid of (threshold, sigma)-like parameters is defined (e.g. min_threshold to max_threshold, with points_per_dimension controlling resolution).
2. Clustering**:** For each parameter combination, DBSCAN is run. Cluster labels are obtained.
3. Similarity score**:** A similarity score (e.g. based on silhouette or parameter sensitivity) is computed for each combination.
4. Selection**:** The combination that maximises the score (with optional decay for robustness) is chosen. The corresponding labels and chosen parameters are returned.

Parameters

- **threshold**: This is the baseline DBSCAN minPts-like value (minimum neighbours to treat a point as dense/core).

- **points_per_dimension**: This controls how finely FINDER samples the parameter space.
- **algorithm**: This picks the clustering backend used for each tested parameter pair, e.g. "dbscan" or "DbscanLoop".
- **min_threshold, max_threshold**: Range of the threshold parameter in the search.
- **decay**: This is the selection sensitivity for choosing the final point on the score-optima curve.

5.8 KNN Analysis of Cluster Centroids (Optional)

When clustering.use_cluster_knn is true, a second pass of nearest-neighbour analysis is run on cluster statistics (centroids and areas), not on raw localizations. This characterises relationships between clusters. For example, the distance from a first-channel cluster to the nearest second channel cluster.

5.8.1 Methodology

- Input: Cluster statistics Data-frames (one per channel), with noise (cluster -1) removed and cluster IDs prefixed by channel (e.g. ch0_0, ch1_0) to avoid collisions. These are concatenated into a single table with columns including x_center, y_center, area_nm2, and cluster.
- Cluster KNN: analyze_nearest_neighbors() is called with this combined stats table, using x_col='x_center', y_col='y_center', and is_clustered=True. Intra- and inter-channel distances between cluster centroids are computed (and optionally mean within radius). Histograms and CSVs are saved with a cluster_ prefix; results are merged into the nearest-cluster DataFrame (e.g. columns nn_distance_to_<other_channel_label>_nm).
- All plots are saved in the clustering output directory. Summary statistics from cluster KNN are appended to the pipeline’s summary for reporting.

5.9 Colocalization Analysis

Colocalization quantifies the spatial association between two molecular species (two channels). The pipeline implements two complementary methods: Coordinate-Based colocalization (CBC) and Relative Enrichment (RE). This step runs only when two channels are present, and colocalization use is true.

5.9.1 Coordinate-Based Colocalization (CBC)

Purpose: For each localization in one channel, measure the distance to the nearest localization in the other channel. The fraction of points within a given radius summarises spatial overlap [10].

Methodology

- Data preparation: The combined localization DataFrame is split by channelIndex into two channel Data-frames. Columns are renamed to Locan conventions (position_x, position_y). Each subset is converted to a locan.LocData object.
- Bidirectional CBC**:** CBC is computed in both directions: Ch1→Ch2 (primary Ch1, reference Ch2) and Ch2→Ch1 (primary Ch2, reference Ch1). For each direction, Locan's CoordinateBasedColocalization is called with the chosen radius and n_steps (number of radius steps for a sweep).
- Output: For each direction, the primary channel's coordinates and the resulting colocalization values are stored. These are used for plotting (e.g. scatter or radius curves) and summary statistics.

Parameters

- **radius**: This is the maximum spatial scale considered for CBC. Think of it as: "Up to what distance should I consider two channels potentially co-located?"
- **n_steps**: This sets how many intermediate radius values are evaluated between 0 and radius.

5.10 Relative Enrichment (RE)

Purpose: Quantify whether the primary channel is enriched or depleted relative to the reference channel in local neighbourhoods defined by the reference channel's Voronoi tessellation, compared to a random expectation [9]. Relative enrichment does not have a separate YAML subsection. It is calculated whenever colocalization.use is enabled for dual-channel data.

Methodology

1. Voronoi tessellation: In relative_enrichment(), the reference channel's localizations are used to build a 2D Voronoi diagram. Each reference point has a Voronoi cell (region). Only finite regions are kept; a boolean index marks valid cells.
2. Primary points per region: For each valid reference cell, the number of primary channel localizations falling inside that cell is counted (using polygon containment and a cKDTree to

limit the primary points queried). This gives, per reference localization, a count n_points of primary localizations in its neighbourhood, the cell area, and the first-order mean distance to neighbouring reference points.

3. Relative enrichment per localization: In `compute_relative_enrichment()`, an expected count is derived from the total number of primary-channel points and the ratio of the region area to the total area. The ratio of observed to expected counts gives a per-reference-localization RE value. Values greater than 1 indicate enrichment of the primary channel near that reference point; values below 1 indicate depletion.

5.11 Summary Statistics and Reporting

Throughout the pipeline, summary statistics are collected and appended to shared output structures, allowing a single experiment run to be summarised in tables or downstream scripts.

- Processing summary: After each major step (initial, after_drift_correction, after_filtering, after_temporal_grouping, after_cell_detection), metrics (e.g. counts, basic stats) are collected. When lp_calculation.use is true, stage CSVs also contain LP_calculated.
- Clustering summary: Histogram and cluster-stat plot results are passed to append_clustering_summary_stats() and saved in the same output area.
- Nearest neighbour summary: Results from localization-level nearest neighbour and (when enabled) cluster-level KNN are passed to append_nn_summary_stats().
- Colocalization summary: CBC and RE summary data frames or statistics are passed to append_coloc_summary_stats().
- Exact file names and formats are defined in utils/utils.py and the pipeline; the summaries aggregate per-step and per-channel information for easy comparison across experiments or conditions.

5.12 Export of Results

The final step consolidates all intermediate data and figures into the experiment's output folder and creates a single archive for sharing or archiving.

5.12.1 What happens in this step?

- Stacked histograms: For each of the metrics lp, bg, sx, sy, and pvalue, create_stacked_histogram() is called. It looks for the step-specific histogram images (initial, after_drift_correction, after_sigma_filter, after_photons_count_filter, after_localization_precision_filter, after_pvalue_filter, after_temporal_grouping, after_cell_detection) in the temporary directory structure. For each metric, it stacks these images vertically into a single image and saves it as e.g. {metric}_hist_all_steps.png in the temp tree. Only the steps that produced a file are included.
- Consolidation of Files
    - All CSV files from the temp tree are moved into temp/test_data/ via move_all_files().

    - CSVs from the sub-folders drift_corrected, filtered, temporal_grouped, cell_detected, and from quantification_results/clustering and quantification_results/distance_based are moved into temp/test_data/.
    - All images (e.g. .png) under the temp tree are moved into temp/test_images/ via move_all_images(), including images from quantification_results/clustering, quantification_results/distance_based, and quantification_results/colocalization.
- ZIP archive: The entire temp directory is zipped using zip_and_rename_folder(). The archive name is the experiment base name (e.g. derived from the input file name), and the archive is written to data/output/. If a file with that name already exists, a unique name is generated (suffix with a letter or number) to avoid overwriting.
- After export, the pipeline considers the run complete for that file. The temp directory is later cleared by the main entry point when all files have been processed. In batch mode, a failure in one input CSV (e.g., a missing x (nm) column) marks only that file as failed, while processing continues for the remaining files. The overall process exits with code 1 if any file fails. A batch-level run_summary is written to data/output/.

5.13 Output Layout (per experiment)

- Experiment archive: data/output/*.zip — Each experiment produces a ZIP archive containing its output data and figures.
- Batch summaries:
    - data/output/run_summary.txt and data/output/run_summary.json — Record the status of each file, failure reasons (if any), processing duration, and a summary of filtering applied or skipped across the batch.
    - data/output/processing_summary_stats.csv — Always generated and appended to across runs.
    - data/output/nn_summary_stats.csv, data/output/clustering_summary_stats.csv, and data/output/coloc_summary_stats.csv — Optionally generated and appended to across runs to support cross-experiment comparisons.
- Contents of the zip archive:
    - test_data/ — Contains all CSV outputs, including initial, drift-corrected, filtered, temporally grouped, cell-detected, clustering, distance-based, and colocalization results, as well as any summary CSVs moved into this directory. Intermediate CSVs may include the LP_calculated column when lp_calculation.use is enabled. The filtering_report.json file is included with the filtered outputs.
    - test_images/ — Contains generated figures, including step-wise and stacked histograms, cluster and nearest-neighbour (NN) plots, CBC and RE figures, cell-detection images, and Voronoi diagrams.
- Portability: Extracting the ZIP archive reproduces the complete set of data and figures for the corresponding experiment in a portable directory structure.

6. Post-processing Scripts

6.1 merge-csv

This command merges CSV outputs from multiple experiments. It scans output folders, groups files by cell line and channel, normalises channel-name variants, and writes consolidated summary CSVs. Separate outputs are produced for cluster statistics, intra-channel results, inter-channel results, and mean-distance measurements, including inter-channel mean distances. Merged files are written to `data/output/merged_channel_data/`.

6.2 save-napari-visualizations

This command automates figure generation. It loads the project configuration and metadata, identifies the channels associated with each dataset, opens the stage-specific CSV files in each output folder, saves Napari screenshots for each processing stage, and combines them into a composite image. It also creates cluster visualizations when clustering results are available.

6.3 visualization-steps

This command provides a step-by-step viewer. It lists the available pipeline stages for the current experiment in the configured output folder, lets the user select a stage from a terminal menu, and opens that stage in Napari. It can be used to inspect the initial, drift-corrected, filtered, temporally grouped, and cell-detected outputs individually. An output folder containing the stage CSV files must already exist, for example after extracting a result archive.

7. References

[1] van de Linde S, Löschberger A, Klein T, Heidbreder M, Wolter S, Heilemann M, and Sauer M (2011). Direct stochastic optical reconstruction microscopy with standard fluorescent probes. Nature Protocols, 6, 991–1009. https://doi.org/10.1038/nprot.2011.336

[2] Thompson RE, Larson DR, and Webb WW (2002) Precise nanometer localization analysis for individual fluorescent probes, Biophysical Journal 82, 2775–2783, doi: 10.1016/S0006-3495(02)75618-X

[3] Schnitzbauer J, Strauss MT, Schlichthaerle T, Schueder F, and Jungmann R (2017). Super-Resolution Microscopy with DNA-PAINT. Nature Protocols, 12, 1198–1228. https://doi.org/10.1038/nprot.2017.024

[4] Nino DF, Milstein JN (2021) Estimating the dynamic range of quantitative single-molecule localization microscopy. Biophys J.120, 3901-3910, doi: 10.1016/j.bpj.2021.08.024

[5] How-to-guide: Temporal grouping tool, ONI Help Center, Jun. 13, 2025. [Online]. Available: https://help.oni.bio/portal/en/kb/articles/how-to-guide-temporal-grouping-tool-13-6-2025 (accessed Mar. 4, 2026).

[6] Ester M, Kriegel H-P, Sander J, and Xu X (1996) A density-based algorithm for discovering clusters in large spatial databases with noise. *Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 226–231.

[7] Campello RJGB, Moulavi D, Zimek A, and Sander J (2015) Hierarchical density estimates for data clustering, visualisation, and outlier detection. ACM Transactions on Knowledge Discovery from Data 10, 1–51, doi: 10.1145/2733381

[8] Verzelli P, Nold A, Sun C, Heilemann M, Schuman EM, Tchumatchenko T (2022) Unbiased choice of global clustering parameters for single-molecule localization microscopy. Scientific Reports 12, 22561, doi: 10.1038/s41598-022-27074-1

[9] Ejdrup AL, Lycas MD, Lorenzen N, Konomi A, Herborg F, Madsen KL, Gether U (2022) A density-based enrichment measure for assessing colocalization in single-molecule localization microscopy data. Nat Commun 13, 4388, doi: 10.1038/s41467-022-32064-y

[10] Doose S (2022) LOCAN: a python library for analysing single-molecule localization microscopy data, Bioinformatics 38, 2670–2672, doi: 10.1093/bioinformatics/btac160

[11] Mortensen KI, Churchman LS, Spudich JA, Flyvbjerg H (2010) Optimized localization analysis for single-molecule tracking and super-resolution microscopy. Nature Methods 7, 377–381, doi: 10.1038/nmeth.1447